FANUC Robot series

R-30*i*B Plus, R-30*i*B Mate Plus,
R-30*i*B Compact Mate Plus CONTROLLER
Version 9.10 and later

HMI Device Communication (with OPC UA or MODBUS/TCP) OPERATOR'S MANUAL

This manual is to be used in conjunction with the documentation provided by the HMI device manufacturer.

MARUCHOPM05191E REV A

Copyrights and Trademarks

The descriptions and specifications contained in this manual were in effect at the time this manual was approved for printing. FANUC America Corporation, hereinafter referred to as FANUC, reserves the right to discontinue models at any time or to change specifications or design without notice and without incurring obligations.

FANUC manuals present descriptions, specifications, drawings, schematics, bills of material, parts, connections and/or procedures for installing, disassembling, connecting, operating and programming FANUC products and/or systems. Such systems consist of robots, extended axes, robot controllers, application software, the KAREL® programming language, INSIGHT® vision equipment, and special tools.

FANUC recommends that only persons who have been trained in one or more approved FANUC Training Course(s) be permitted to install, operate, use, perform procedures on, repair, and/or maintain FANUC products and/or systems and their respective components. Approved training necessitates that the courses selected be relevant to the type of system installed and application performed at the customer site.

⚠ WARNING

This equipment generates, uses, and can radiate radiofrequency energy and if not installed and used in accordance with the instruction manual, may cause interference to radio communications. As temporarily permitted by regulation, it has not been tested for compliance with the limits for Class A computing devices pursuant to Subpart J and Part 15 of FCC Rules, which are designed to provide reasonable protection against such interference. Operation of the equipment in a residential area is likely to cause interference, in which case the user, at his own expense, will be required to take whatever measure may be required to correct the interference.

FANUC conducts courses on its systems and products on a regularly scheduled basis at the company's world headquarters in Rochester Hills, Michigan. For additional information contact

FANUC America Corporation
Training Department
3900 W. Hamlin Road
Rochester Hills, Michigan 48309-3253
www.fanucamerica.com

For customer assistance, including Technical Support, Service, Parts & Part Repair, and Marketing Requests, contact the Customer Resource Center, 24 hours a day, at 888-FANUC-US (888-326-8287).

Send your comments and suggestions about this manual to:
product.documentation@fanucamerica.com

Patents

One or more of the following U.S. patents might be related to the FANUC products described in this manual.

FANUC America Corporation Patent List

4,630,567 4,639,878 4,707,647 4,708,175 4,708,580 4,942,539 4,984,745 5,238,029
5,239,739 5,272,805 5,293,107 5,293,911 5,331,264 5,367,944 5,373,221 5,421,218
5,434,489 5,644,898 5,670,202 5,696,687 5,737,218 5,823,389 5,853,027 5,887,800
5,941,679 5,959,425 5,987,726 6,059,092 6,064,168 6,070,109 6,086,294 6,122,062
6,147,323 6,204,620 6,243,621 6,253,799 6,285,920 6,313,595 6,325,302 6,345,818
6,356,807 6,360,143 6,378,190 6,385,508 6,425,177 6,477,913 6,490,369 6,518,980
6,540,104 6,541,757 6,560,513 6,569,258 6,612,449 6,703,079 6,705,361 6,726,773
6,768,078 6,845,295 6,945,483 7,149,606 7,149,606 7,211,978 7,266,422 7,399,363

FANUC CORPORATION Patent List

4,571,694 4,626,756 4,700,118 4,706,001 4,728,872 4,732,526 4,742,207 4,835,362
4,894,596 4,899,095 4,920,248 4,931,617 4,934,504 4,956,594 4,967,125 4,969,109
4,970,370 4,970,448 4,979,127 5,004,968 5,006,035 5,008,834 5,063,281 5,066,847
5,066,902 5,093,552 5,107,716 5,111,019 5,130,515 5,136,223 5,151,608 5,170,109
5,189,351 5,267,483 5,274,360 5,292,066 5,300,868 5,304,906 5,313,563 5,319,443
5,325,467 5,327,057 5,329,469 5,333,242 5,337,148 5,371,452 5,375,480 5,418,441
5,432,316 5,440,213 5,442,155 5,444,612 5,449,875 5,451,850 5,461,478 5,463,297
5,467,003 5,471,312 5,479,078 5,485,389 5,485,552 5,486,679 5,489,758
5,493,192 5,504,766 5,511,007 5,520,062 5,528,013 5,532,924 5,548,194 5,552,687
5,558,196 5,561,742 5,570,187 5,570,190 5,572,103 5,581,167 5,582,750 5,587,635
5,600,759 5,608,299 5,608,618 5,624,588 5,630,955 5,637,969 5,639,204 5,641,415
5,650,078 5,658,121 5,668,628 5,687,295 5,691,615 5,698,121 5,708,342 5,715,375
5,719,479 5,727,132 5,742,138 5,742,144 5,748,854 5,749,058 5,760,560 5,773,950
5,783,922 5,799,135 5,812,408 5,841,257 5,845,053 5,872,894 5,887,122 5,911,892
5,912,540 5,920,678 5,937,143 5,980,082 5,983,744 5,987,591 5,988,850 6,023,044
6,032,086 6,040,554 6,059,169 6,088,628 6,097,169 6,114,824 6,124,693
6,140,788 6,141,863 6,157,155 6,160,324 6,163,124 6,177,650 6,180,898 6,181,096
6,188,194 6,208,105 6,212,444 6,219,583 6,226,181 6,236,011 6,236,896 6,250,174
6,278,902 6,279,413 6,285,921 6,298,283 6,321,139 6,324,443 6,328,523 6,330,493
6,340,875 6,356,671 6,377,869 6,382,012 6,384,371 6,396,030 6,414,711 6,424,883
6,431,018 6,434,448 6,445,979 6,459,958 6,463,358 6,484,067 6,486,629 6,507,165
6,654,666 6,665,588 6,680,461 6,696,810 6,728,417 6,763,284 6,772,493 6,845,296
6,853,881 6,888,089 6,898,486 6,917,837 6,928,337 6,965,091 6,970,802 7,038,165
7,069,808 7,084,900 7,092,791 7,133,747 7,143,100 7,149,602 7,131,848
7,161,321 7,171,041 7,174,234 7,173,213 7,177,722 7,177,439 7,181,294 7,181,313
7,280,687 7,283,661 7,291,806 7,299,713 7,315,650 7,324,873 7,328,083 7,330,777
7,333,879 7,355,725 7,359,817 7,373,220 7,376,488 7,386,367 7,464,623 7,447,615
7,445,260 7,474,939 7,486,816 7,495,192 7,501,778 7,502,504 7,508,155 7,512,459
7,525,273 7,526,121

Conventions

WARNING

Information appearing under the "WARNING" caption concerns the protection of personnel. It is boxed and bolded to set it apart from the surrounding text.

CAUTION

Information appearing under the "CAUTION" caption concerns the protection of equipment, software, and data. It is boxed and bolded to set it apart from the surrounding text.

Note Information appearing next to NOTE concerns related information or useful hints.

Safety

FANUC America Corporation is not and does not represent itself as an expert in safety systems, safety equipment, or the specific safety aspects of your company and/or its work force. It is the responsibility of the owner, employer, or user to take all necessary steps to guarantee the safety of all personnel in the workplace.

The appropriate level of safety for your application and installation can be best determined by safety system professionals. FANUC America Corporation therefore, recommends that each customer consult with such professionals in order to provide a workplace that allows for the safe application, use, and operation of FANUC America Corporation systems.

According to the industry standard ANSI/RIA R15-06, the owner or user is advised to consult the standards to ensure compliance with its requests for Robotics System design, usability, operation, maintenance, and service. Additionally, as the owner, employer, or user of a robotic system, it is your responsibility to arrange for the training of the operator of a robot system to recognize and respond to known hazards associated with your robotic system and to be aware of the recommended operating procedures for your particular application and robot installation.

Ensure that the robot being used is appropriate for the application. Robots used in classified (hazardous) locations must be certified for this use.

FANUC America Corporation therefore, recommends that all personnel who intend to operate, program, repair, or otherwise use the robotics system be trained in an approved FANUC America Corporation training course and become familiar with the proper operation of the system. Persons responsible for programming the system–including the design, implementation, and debugging of application programs–must be familiar with the recommended programming procedures for your application and robot installation.

The following guidelines are provided to emphasize the importance of safety in the workplace.

CONSIDERING SAFETY FOR YOUR ROBOT INSTALLATION

Safety is essential whenever robots are used. Keep in mind the following factors with regard to safety:

- The safety of people and equipment
- Use of safety enhancing devices
- Techniques for safe teaching and manual operation of the robot(s)
- Techniques for safe automatic operation of the robot(s)
- Regular scheduled inspection of the robot and workcell
- Proper maintenance of the robot

Keeping People Safe

The safety of people is always of primary importance in any situation. When applying safety measures to your robotic system, consider the following:

- External devices
- Robot(s)
- Tooling
- Workpiece

Using Safety Enhancing Devices

Always give appropriate attention to the work area that surrounds the robot. The safety of the work area can be enhanced by the installation of some or all of the following devices:

- Safety fences, barriers, or chains
- Light curtains
- Interlocks
- Pressure mats
- Floor markings
- Warning lights
- Mechanical stops
- EMERGENCY STOP buttons
- DEADMAN switches

Setting Up a Safe Workcell

A safe workcell is essential to protect people and equipment. Observe the following guidelines to ensure that the workcell is set up safely. These suggestions are intended to supplement and not replace existing federal, state, and local laws, regulations, and guidelines that pertain to safety.

- Sponsor your personnel for training in approved FANUC America Corporation training course(s) related to your application. Never permit untrained personnel to operate the robots.
- Install a lockout device that uses an access code to prevent unauthorized persons from operating the robot.
- Use anti–tie–down logic to prevent the operator from bypassing safety measures.
- Arrange the workcell so the operator faces the workcell and can see what is going on inside the cell.
- Clearly identify the work envelope of each robot in the system with floor markings, signs, and special barriers. The work envelope is the area defined by the maximum motion range of the robot, including any tooling attached to the wrist flange that extend this range.
- Position all controllers outside the robot work envelope.
- Never rely on software or firmware based controllers as the primary safety element unless they comply with applicable current robot safety standards.
- Mount an adequate number of EMERGENCY STOP buttons or switches within easy reach of the operator and at critical points inside and around the outside of the workcell.

- Install flashing lights and/or audible warning devices that activate whenever the robot is operating, that is, whenever power is applied to the servo drive system. Audible warning devices shall exceed the ambient noise level at the end–use application.
- Wherever possible, install safety fences to protect against unauthorized entry by personnel into the work envelope.
- Install special guarding that prevents the operator from reaching into restricted areas of the work envelope.
- Use interlocks.
- Use presence or proximity sensing devices such as light curtains, mats, and capacitance and vision systems to enhance safety.
- Periodically check the safety joints or safety clutches that can be optionally installed between the robot wrist flange and tooling. If the tooling strikes an object, these devices dislodge, remove power from the system, and help to minimize damage to the tooling and robot.
- Make sure all external devices are properly filtered, grounded, shielded, and suppressed to prevent hazardous motion due to the effects of electro–magnetic interference (EMI), radio frequency interference (RFI), and electro–static discharge (ESD).
- Make provisions for power lockout/tagout at the controller.
- Eliminate *pinch points*. Pinch points are areas where personnel could get trapped between a moving robot and other equipment.
- Provide enough room inside the workcell to permit personnel to teach the robot and perform maintenance safely.
- Program the robot to load and unload material safely.
- If high voltage electrostatics are present, be sure to provide appropriate interlocks, warning, and beacons.
- If materials are being applied at dangerously high pressure, provide electrical interlocks for lockout of material flow and pressure.

Staying Safe While Teaching or Manually Operating the Robot

Advise all personnel who must teach the robot or otherwise manually operate the robot to observe the following rules:

- Never wear watches, rings, neckties, scarves, or loose clothing that could get caught in moving machinery.
- Know whether or not you are using an intrinsically safe teach pendant if you are working in a hazardous environment.
- Before teaching, visually inspect the robot and work envelope to make sure that no potentially hazardous conditions exist. The work envelope is the area defined by the maximum motion range of the robot. These include tooling attached to the wrist flange that extends this range.
- The area near the robot must be clean and free of oil, water, or debris. Immediately report unsafe working conditions to the supervisor or safety department.
- FANUC America Corporation recommends that no one enter the work envelope of a robot that is on, except for robot teaching operations. However, if you must enter the work envelope, be sure all safeguards are in place, check the teach pendant DEADMAN switch for proper operation, and place the robot in teach mode. Take the teach pendant with you, turn it on, and be prepared to release the DEADMAN switch. Only the person with the teach pendant should be in the work envelope.

WARNING

Never bypass, strap, or otherwise deactivate a safety device, such as a limit switch, for any operational convenience. Deactivating a safety device is known to have resulted in serious injury and death.

- Know the path that can be used to escape from a moving robot; make sure the escape path is never blocked.
- Isolate the robot from all remote control signals that can cause motion while data is being taught.
- Test any program being run for the first time in the following manner:

WARNING

Stay outside the robot work envelope whenever a program is being run. Failure to do so can result in injury.

  - Using a low motion speed, single step the program for at least one full cycle.
  - Using a low motion speed, test run the program continuously for at least one full cycle.
  - Using the programmed speed, test run the program continuously for at least one full cycle.
- Make sure all personnel are outside the work envelope before running production.

Staying Safe During Automatic Operation

Advise all personnel who operate the robot during production to observe the following rules:

- Make sure all safety provisions are present and active.
- Know the entire workcell area. The workcell includes the robot and its work envelope, plus the area occupied by all external devices and other equipment with which the robot interacts.
- Understand the complete task the robot is programmed to perform before initiating automatic operation.
- Make sure all personnel are outside the work envelope before operating the robot.
- Never enter or allow others to enter the work envelope during automatic operation of the robot.
- Know the location and status of all switches, sensors, and control signals that could cause the robot to move.
- Know where the EMERGENCY STOP buttons are located on both the robot control and external control devices. Be prepared to press these buttons in an emergency.
- Never assume that a program is complete if the robot is not moving. The robot could be waiting for an input signal that will permit it to continue its activity.
- If the robot is running in a pattern, do not assume it will continue to run in the same pattern.

- Never try to stop the robot, or break its motion, with your body. The only way to stop robot motion immediately is to press an EMERGENCY STOP button located on the controller panel, teach pendant, or emergency stop stations around the workcell.

Staying Safe During Inspection

When inspecting the robot, be sure to

- Turn off power at the controller.
- Lock out and tag out the power source at the controller according to the policies of your plant.
- Turn off the compressed air source and relieve the air pressure.
- If robot motion is not needed for inspecting the electrical circuits, press the EMERGENCY STOP button on the operator panel.
- Never wear watches, rings, neckties, scarves, or loose clothing that could get caught in moving machinery.
- If power is needed to check the robot motion or electrical circuits, be prepared to press the EMERGENCY STOP button, in an emergency.
- Be aware that when you remove a servomotor or brake, the associated robot arm will fall if it is not supported or resting on a hard stop. Support the arm on a solid support before you release the brake.

Staying Safe During Maintenance

When performing maintenance on your robot system, observe the following rules:

- Never enter the work envelope while the robot or a program is in operation.
- Before entering the work envelope, visually inspect the workcell to make sure no potentially hazardous conditions exist.
- Never wear watches, rings, neckties, scarves, or loose clothing that could get caught in moving machinery.
- Consider all or any overlapping work envelopes of adjoining robots when standing in a work envelope.
- Test the teach pendant for proper operation before entering the work envelope.
- If it is necessary for you to enter the robot work envelope while power is turned on, you must be sure that you are in control of the robot. Be sure to take the teach pendant with you, press the DEADMAN switch, and turn the teach pendant on. Be prepared to release the DEADMAN switch to turn off servo power to the robot immediately.
- Whenever possible, perform maintenance with the power turned off. Before you open the controller front panel or enter the work envelope, turn off and lock out the 3–phase power source at the controller.
- Be aware that when you remove a servomotor or brake, the associated robot arm will fall if it is not supported or resting on a hard stop. Support the arm on a solid support before you release the brake.

⚠WARNING

Lethal voltage is present in the controller WHENEVER IT IS CONNECTED to a power source. Be extremely careful to avoid electrical shock. HIGH VOLTAGE IS PRESENT at the input side whenever the controller is connected to a power source. Turning the disconnect or circuit breaker to the OFF position removes power from the output side of the device only.

- Release or block all stored energy. Before working on the pneumatic system, shut off the system air supply and purge the air lines.
- Isolate the robot from all remote control signals. If maintenance must be done when the power is on, make sure the person inside the work envelope has sole control of the robot. The teach pendant must be held by this person.
- Make sure personnel cannot get trapped between the moving robot and other equipment. Know the path that can be used to escape from a moving robot. Make sure the escape route is never blocked.
- Use blocks, mechanical stops, and pins to prevent hazardous movement by the robot. Make sure that such devices do not create pinch points that could trap personnel.

⚠WARNING

Do not try to remove any mechanical component from the robot before thoroughly reading and understanding the procedures in the appropriate manual. Doing so can result in serious personal injury and component destruction.

- Be aware that when you remove a servomotor or brake, the associated robot arm will fall if it is not supported or resting on a hard stop. Support the arm on a solid support before you release the brake.
- When replacing or installing components, make sure dirt and debris do not enter the system.
- Use only specified parts for replacement. To avoid fires and damage to parts in the controller, never use nonspecified fuses.
- Before restarting a robot, make sure no one is inside the work envelope; be sure that the robot and all external devices are operating normally.

KEEPING MACHINE TOOLS AND EXTERNAL DEVICES SAFE

Certain programming and mechanical measures are useful in keeping the machine tools and other external devices safe. Some of these measures are outlined below. Make sure you know all associated measures for safe use of such devices.

Programming Safety Precautions

Implement the following programming safety measures to prevent damage to machine tools and other external devices.

- Back–check limit switches in the workcell to make sure they do not fail.
- Implement ‘‘failure routines” in programs that will provide appropriate robot actions if an external device or another robot in the workcell fails.

- Use *handshaking* protocol to synchronize robot and external device operations.
- Program the robot to check the condition of all external devices during an operating cycle.

Mechanical Safety Precautions

Implement the following mechanical safety measures to prevent damage to machine tools and other external devices.

- Make sure the workcell is clean and free of oil, water, and debris.
- Use DCS (Dual Check Safety), software limits, limit switches, and mechanical hardstops to prevent undesired movement of the robot into the work area of machine tools and external devices.

KEEPING THE ROBOT SAFE

Observe the following operating and programming guidelines to prevent damage to the robot.

Operating Safety Precautions

The following measures are designed to prevent damage to the robot during operation.

- Use a low override speed to increase your control over the robot when jogging the robot.
- Visualize the movement the robot will make before you press the jog keys on the teach pendant.
- Make sure the work envelope is clean and free of oil, water, or debris.
- Use circuit breakers to guard against electrical overload.

Programming Safety Precautions

The following safety measures are designed to prevent damage to the robot during programming:

- Establish *interference zones* to prevent collisions when two or more robots share a work area.
- Make sure that the program ends with the robot near or at the home position.
- Be aware of signals or other operations that could trigger operation of tooling resulting in personal injury or equipment damage.
- In dispensing applications, be aware of all safety guidelines with respect to the dispensing materials.

NOTE: Any deviation from the methods and safety practices described in this manual must conform to the approved standards of your company. If you have questions, see your supervisor.

TABLE OF CONTENTS

1 OVERVIEW

The HMI Device function allows communication with external industrial devices that are connected via RS-232-C or Ethernet via MODBUS/TCP or OPC UA. Various robot data, such as DI, DO, Registers, etc. can be read from or written to by external industrial devices.

This function operates as either:

Modbus slave, communicating with HMI device etc. that is the MODBUS master

Or,

OPC UA Server, communicating with OPC UA Clients.

HMI device can design the screen by putting lumps, switches and so on. The assignment of robot data to the lumps, switches and so on can be defined by user. For example, if robot register is assigned to the display panel on HMI device, the register value can be displayed on the screen of HMI device.

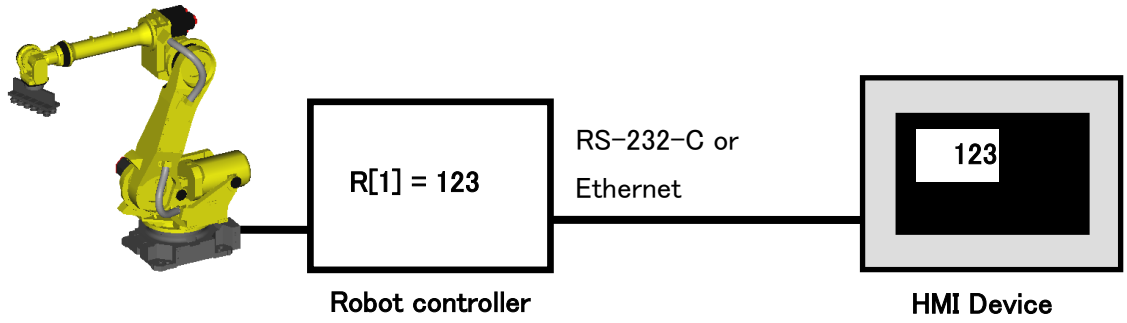


Figure 1-1. Connection Overview

When "Standard setting (R651)" is selected, this function is standard. When "North America Setting (R650)" is selected, "HMI device (R553)" option is needed to use this function.

Modbus slave function is supported from version 7DC2 series P07 and all versions after 7DC2 series.

OPC UA server function is supported from 7DF1 series P19 or after and all versions after 7DF1 series.
OPC UA server function supports OPC UA publisher so client can use subscriber. (Minimum data acquiring interval is 100ms.)

⚠ WARNING

This function checks the received data and performs the proper process only when the robot controller receives the data sent from HMI device etc. This function cannot detect the situation that the communication from HMI device etc. is stopped accidentally. Therefore, this function must not be used to transfer the safety data. For example, when HMI device etc. sends STOP request to robot, there is no guarantee that the robot is stopped by the request. It is very dangerous to make safety system by using this function.

NOTE

When this function is used to communicate with HMI device etc., please test the whole robot system including the HMI device etc.

Connection tested HMI devices

Schneider Electric Japan Holdings Ltd. GP4000 Series

Set followings to the “Manufacturer” and “Series”.

RS-232-C : "Manufacturer: Modbus-IDA", "Series: MODBUS RTU SIO master"

Ethernet: "Manufacturer: Modbus-IDA", " Series: MODBUS TCP master"

KEYENCE CORPORATION VT3 Series

Set the following to the “Manufacturer” and “Model”.

RS-232-C: "Manufacturer: MODBUS protocol", "Model: RTU mode (1:N)"

Ethernet: "Manufacturer: MODBUS protocol", "Model: MODBUS/TCP(Ethernet)"

2 CONNECTION OF HMI DEVICE

2.1 Modbus RS-232-C Connection

Cable

The following diagram shows the connection of RS-232-C. Please refer to the manual of the HMI device about the connection of HMI device side.

A-cabinet

The cable of the ROBOT CONTROLLER side is connected to JRS16 or JD17 connector on the main board.

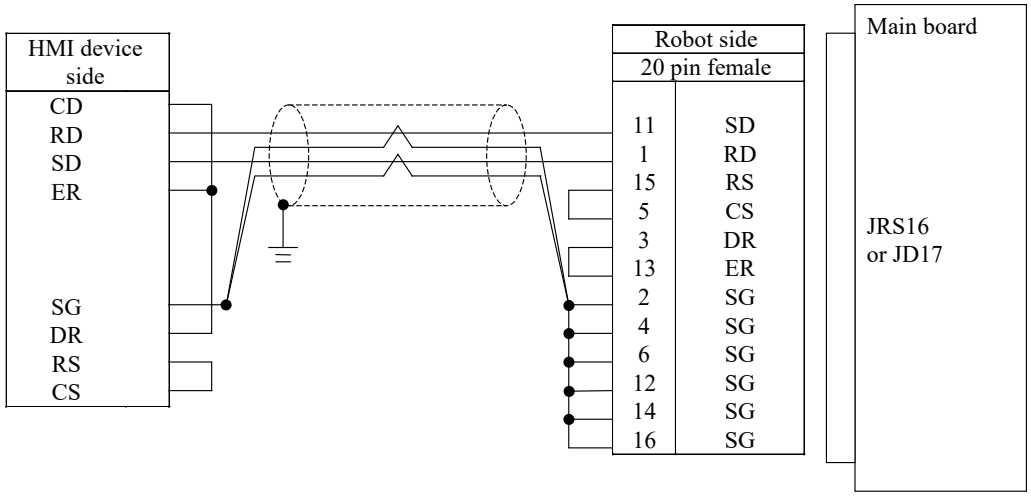


Figure 2-1. Modbus RS-232-C Connection for A-Cabinet

Specification of the connectors and the cable are as following:

- Connector PCR-E20FS (Honda Tsushin Kogyo)
- Housing PCR-V20LA (Honda Tsushin Kogyo), or compatible connector
- Shielding quality Entirely shielded twisted pair cable is recommended.

  For protection against the noise, the twisted pair wire should be used for pairs of RD and SG, SD and SG.

B-cabinet

The D-Sub 25pin is connected to the RS-232-C port in front side of ROBOT CONTROLLER operator panel or JD17 connector on the main board.

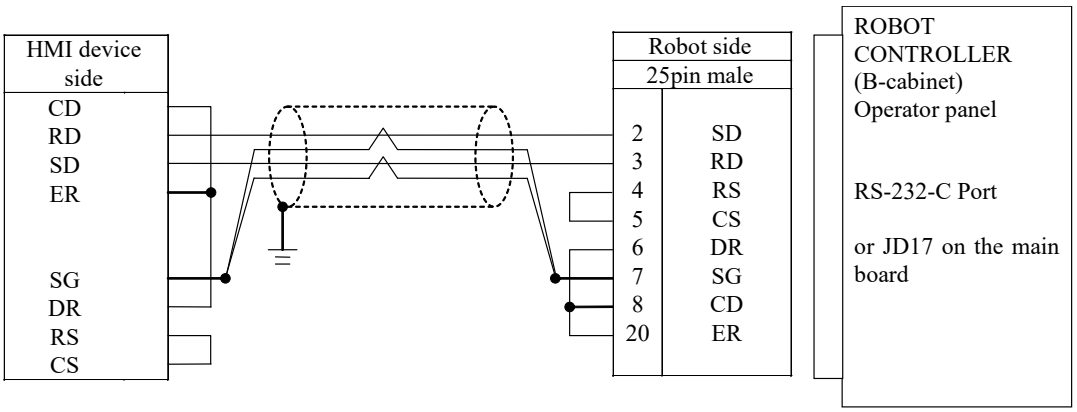


Figure 2-2. Modbus RS-232-C Connection for B-Cabinet

Specification of the connectors and the cable are as following (Operator panel):

- Connector D-Sub25pin male DBM-25P (ANSI/EIA-232)
- Housing DB-C2-J9 (ANSI/EIA-232)
- Shielding quality Entirely shielded twisted pair cable is recommended.

  For protection against the noise, the twisted pair wire should be used for pairs of RD and SG, SD and SG.

R-30iB Mate

The cable of the ROBOT CONTROLLER side is connected to JRS27 connector on the main board.

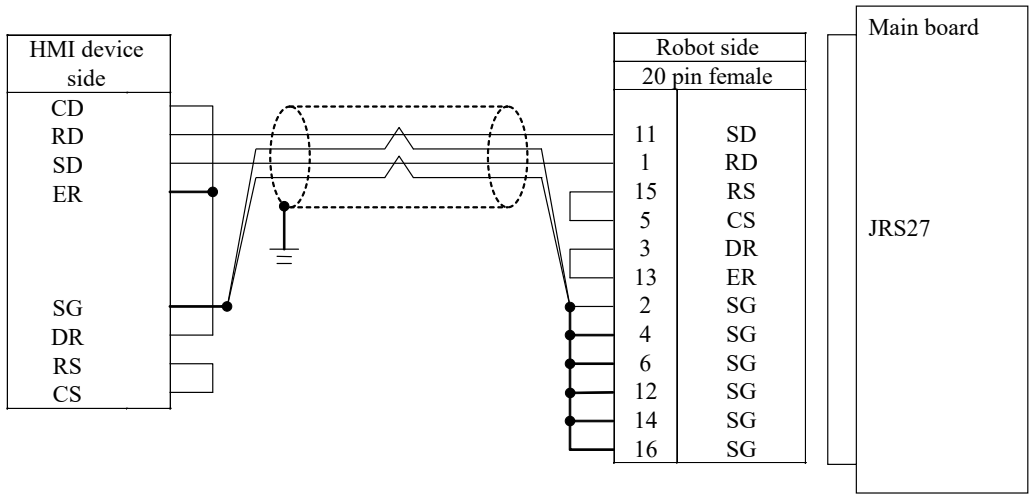


Figure 2-3. Modbus RS-232-C Connection for R-30iB Mate Cabinet

Specification of the connectors are as following:

- Connector PCR-E20FS (Honda Tsushin Kogyo)
- Housing PCR-V20LA (Honda Tsushin Kogyo), or compatible connector
- Shielding quality Entirely shielded twisted pair cable is recommended.

  For protection against the noise, the twisted pair wire should be used for pairs of RD and SG, SD and SG.

Setting of Serial Port

To use HMI device communication via RS-232-C, it is necessary to set "HMI dev(MODBUS)" in Port Init menu on Robot controller.

1 Press the [MENU] key on teach pendant, and select "6 SETUP".

2 Press the F1[TYPE] key, and select "Port Init".

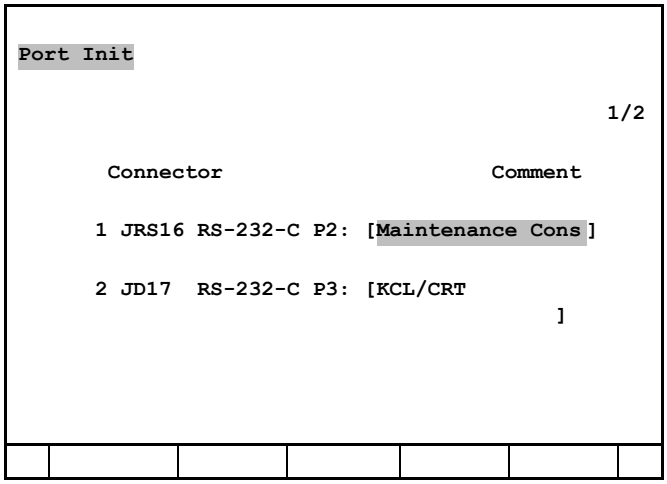


Figure 2-4. "Port Init" Setup Screen

3 Set cursor on "JRS16 RS-232-C P2:" line, and press the F3(DETAIL) key. The detail menu of "JRS16 RS-232-C P2:" is displayed.

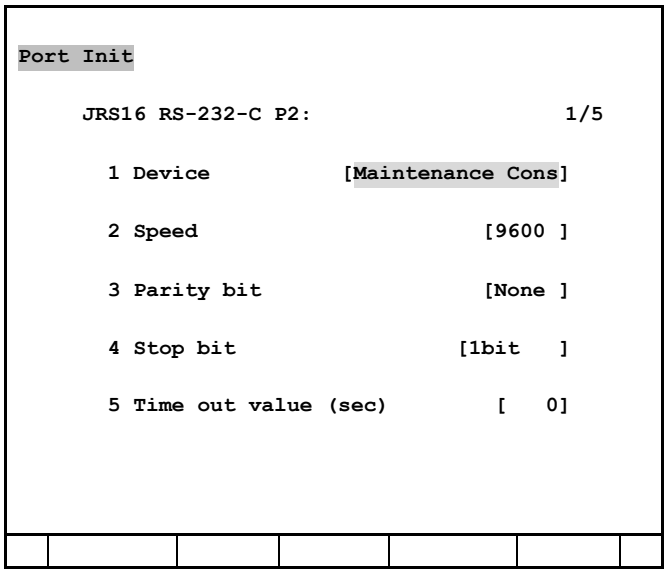


Figure 2-5. "Port Init" Detail Screen

4 Set cursor on "Device" line, and press the F4[CHOICE], and select "HMI dev(MODBUS)" from the list. There are "HMI device" and "HMI dev(MODBUS)" in the list. It is necessary to set "HMI dev(MODBUS)" for Modbus communication. It is not possible to use "HMI device" and "HMI dev(MODBUS)" at the same time.

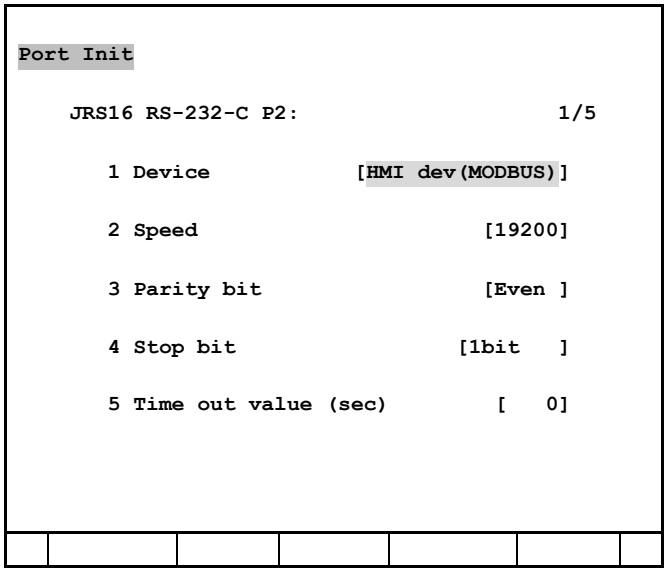


Figure 2-6. Selection of "HMI dev(MODBUS)" from the "Port Init" Detail Screen

5 Setup "Speed", "Parity bit" and "Stop bit" as same with the HMI device.

Setup slave address

The slave address of this function is set in the system variable $SNPX_PARAM.$MODBUS_ADR. The default value is 1. The slave address of Modbus RTU can be changed by this system variable. This function responds to the request for the specified slave address only. The request for the slave address 0 is processed as broadcast.

2.1.1 Modbus/TCP Slave Ethernet Connection

Setting of TCP/IP

To use HMI device communication via MODBUS/TCP Slave Ethernet, it is necessary to setup TCP/IP on Robot controller in advance.

Details on the Ethernet interface and TCP/IP configuration can be found in "SETTING UP TCP/IP" section in the *Internet Options Setup and Operations Manual*.

Setting of MODBUS/TCP Slave Ethernet connection

MODBUS/TCP Slave Ethernet connection is disable by default. To use MODBUS/TCP Slave Ethernet connection, please set the number of HMI devices etc. that are connected at the same time to the system variable $SNPX_PARAM.$NUM_MODBUS (Default is 0). When a MODBUS/TCP Slave Ethernet

connection is requested from HMI devices etc. which would exceed the setting value, the HMI device etc. whose latest communication is the oldest is disconnected automatically.

Unit ID

Robot controller does not use Unit ID in Modbus TCP communication data. Robot controller responds all requests from an HMI device etc. regardless of Unit ID.

Port number

The port number of this function is set in the system variable $SNPX_PARAM.$MODBUS_PORT. The default value is 502. The port number of this function can be changed by this system variable.

Modbus TCP Server (R800)

Modbus TCP Server (R800) option is independent from this function. But when this option is loaded, if $SNPX_PARAM.$NUM_MODBUS is set to 1 or greater value, "PRIO-090 SNPX communication error", "HRTL-048 Address already in use" occurs and MODBUS/TCP Slave Ethernet communication of this function does not work. It is because this function and Modbus TCP Server use the same port number (502).

By changing the port number of this function, both this function and Modbus TCP Server can be used at the same time.

2.1.2 OPC UA Server Ethernet Connection

OPC UA is multi-purpose and flexible industrial communication protocol which enables data exchange between multi-vendor products and different operating systems. This function enables the FANUC robot to connect with OPC UA clients and enables various robot data (such as DI, DO, Registers, and so on) to be read and written. This function works as an OPC UA server and supports communication with external OPC UA clients over Ethernet.

The data model (information model) used for sending and receiving data is based on the Modbus data model of the “HMI DEVICE COMMUNICATION” function.

Limitation

The following limitations exist when using the FANUC robot OPC UA Server.
- Maximum of ten (10) OPC UA clients can communicate simultaneously.
- This function can be used only with CD38A and CD38B RJ45 Ethernet port connections on the robot controller. This function cannot be used with the CD38C RJ45 Ethernet port connection.
- Up to 2 subscriptions can be registered per 1 connection.

- 50 items can be monitored per 1 subscription but minimum data acquiring interval for the item is increased by 100ms per 1 item like 100ms for 1st item, 200ms for 2nd item, 300ms for 3rd item.

Setting of TCP/IP

To use HMI device communication via the FANUC robot OPC UA Server, it is necessary to setup TCP/IP on the Robot controller in advance. Details on the Ethernet interface and TCP/IP configuration can be found in "SETTING UP TCP/IP" section in the *Internet Options Setup and Operations Manual*.

The FANUC robot OPC UA Server can be accessed from an external OPC UA client by entering the robot's OPC UA server URL in the following format:

opc.tcp://robot IP address:4880/FANUC/NanoUaServer

Example:

opc.tcp://172.22.194.222:4880/FANUC/NanoUaServer

The OPC UA client can connect with the robot OPC UA server after setting the above URL in the OPC UA client. Please see the "55.2 MODBUS DATA MODEL" sections for details about assigning and accessing robot data.

3 MODBUS DATA MODEL

3.1 Modbus data model

Modbus bases its data model on a series of tables.

Table 3-1. Listing of Modbus data model nodes

Table	Object type	Type of
Discrete input	Single bit	READ only
Coils	Single bit	READ-WRITE
Input Registers	16-bit word	READ only
Holding Registers	16-bit word	READ-WRITE

3.2 OPC UA Organization of MODBUS data model

This section applies only when using OPC UA client for HMI device communication with the built-in FANUC robot OPC UA Server.

The “Modbus” object nodes can be accessed from the “Root/Object/Modbus” directory. As displayed below.

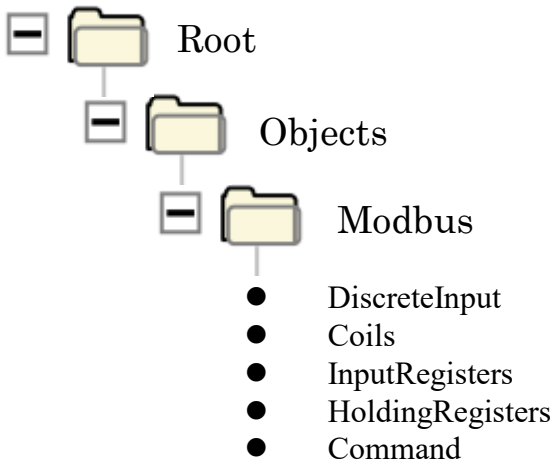


Figure 3-1. OPC UA Organization of MODBUS data model

There are other folders just under root folder but is not used.

The following table describes node and data types of the five (5) leaf nodes under the “Modbus” object.

Table 3-2. Listing of Modbus node data types

Node	Data type
DiscreteInput	Boolean
Coils	Boolean
InputRegisters	UInt16
HoldingRegisters	Uint16
Command	String[]

Leaf nodes other than Command node are array.
The index range should be supported on the OPC UA client to support partial read or write for the leaf nodes.

Please Note: In the following sections, the term *HMI Device etc.* can also refer to an OPC UA client.

3.2.1 Correspondence of Modbus Address to Robot Data

Each “Table” has address area 1 ~ 65535. This function defines the following correspondence of Modbus address to Robot data. The HMI device etc. can read or write the corresponded robot data by accessing Modbus address.
OPC UA clients can read or write the desired robot data by accessing the index which is “Modbus addresses - 1“. Modbus address is described in the following sections.

Table 3-3. Cross Reference of Modbus address to robot data

Table	Address range	Corresponded robot data (a: address)	
Discrete input	1 ~ 10000	Digital input	DI[a]
	10001 ~ 20000	Robot input	RI[a-10000]
	20001 ~ 21000	UOP input	UI[a-20000]
	21001 ~ 21999	UOP output	UO[a-21000]
	22000 ~ 22999	SOP input	SI[a-22000]
	23000 ~ 24000	SOP output	SO[a-23000]
	24001 ~ 25000	Weld interface digital input	WI[a-24000]
	25001 ~ 26000	Weld interface digital output	WO[a-25000]
	26001 ~ 27000	Wire soldering detector input	WSI[a-26000]
	27001 ~ 28000	Wire soldering detector output	WSO[a-27000]
	28001 ~ 65536	Not used	

Table 3-3. Cross Reference of Modbus address to robot data

Coils	1 ~ 10000	Digital output	DO[a]
	10001 ~ 20000	Robot output	RO[a-10000]
	20001 ~ 30000	Flag	F[a-20000]
	30001 ~ 65536	Not used	
Input Registers	1 ~ 1000	Group input	GI[a]
	1001 ~ 2000	Group output	GO[a-1000]
	2001 ~ 3000	Analog input	AI[a-2000]
	3001 ~ 4000	Analog output	AO[a-3000]
	4001 ~ 65536	Not used	
Holding Registers	1 ~ 16384	Robot data is assigned (assigned to R[a] by default)	
	16385 ~ 65536	Not used	

When reading an address that is not used (or the corresponding robot data is not assigned) from an HMI device etc., the read value is always 0. When the address is written from an HMI device etc., the request is ignored. In these cases, this function does not return an error to the HMI device etc.

The address range 1 ~ 16384 of Holding Registers can be assigned to various Robot data. (→ Assignment of Holding Registers)

The HMI device etc. cannot write to the signals corresponded to Discrete input and Input Registers. It is possible to write these signals via Holding Registers by assigning these signals to Holding Registers. (→ Assign I/O data and simulation status)

In this function, address range is specified as 1 ~ 65536. But some HMI devices etc. specify as 0000 ~ FFFF (start from 0 and specified as hexadecimal). In this case, please convert the specified address as subtraction of 1 and change to hexadecimal.

3.2.2 Modbus Function Code

When using MODBUS TCP communication (not OPC UA communication), this function supports the following function codes.

Table 3-4. MODBUS TCP function codes

Function code name	Code
Read Coils	01h
Read Discrete Inputs	02h
Read Holding Registers	03h
Read Input Register	04h
Write Single Coil	05h
Write Single Register	06h
Write Multiple Coils	0Fh
Write Multiple Registers	10h

3.3 ASSIGNMENT OF HOLDING REGISTERS

The address range 1 ~ 16384 (0~16383 when OPC UA) of Holding Registers can be assigned to various Robot data. Each address has 2 bytes (16 bits) memory space. It is possible to assign 32 bit data to the continuous 2 addresses, and to assign string data to the continuous plural addresses.

The following robot data can be assigned to the Holding Registers.

- Register data
- Position resister data
- String register data
- Current position
- Alarm history
- Program execution status (Running program name and line number)
- System variable data
- Comment of Register, Position register, String register and I/O

- I/O data and simulation status
- Integrated PMC address data
- Symbol and comment of Integrated PMC address.

The assignment between Robot data and Holding Registers is defined by setting of $SNPX_ASG. The $SNPX_ASG is 80 arrays of $SNPX_ASG[1]~[80], every element has the following variables. Various robot data can be assigned to the Holding Registers by setting $SNPX_ASG.

Table 3-5. $SNPX_ASG variable descriptions

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. Range: 1~16384
$VAR_NAME	Meaning: String to specify robot data. It specifies the robot data type and index by string. The meaning of this variable is different according to the assigned data type. Please refer to the description in each robot data. Example: R[1] : Register R[1] PR[1] : Position register PR[1] POS[1] : Current position of group 1 robot
$MULTIPLY	Meaning: Multiply for assigned data. It specifies the conversion type from assigned data to Holding Registers. The meaning of this variable is different according to the assigned data type. Please refer to the description in each robot data. Example: When register is assigned and the register value is 123.45, the value of the Holding Registers is the following. When $MULTIPLY is 1, Holding Register is 123. When $MULTIPLY is10, Holding Register is 1235 When $MULTIPLY is 0.1, Holding Register is 12

The following figure shows a example that robot register R[1~4] are assigned to Holding Register address 1 ~ 4 and X,Y,Z of robot position register PR[1] are assigned to Holding Register address 5 ~ 10.

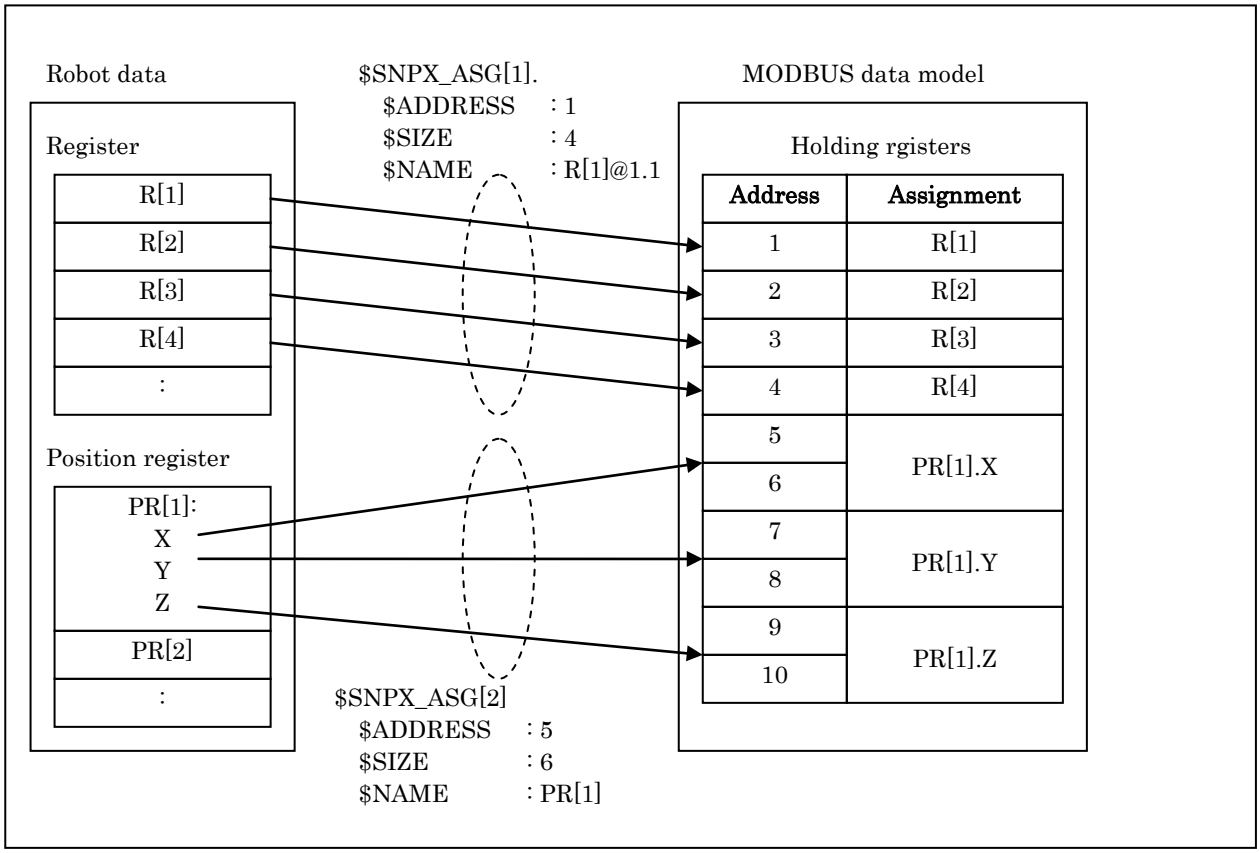


Figure 3-2. Example of Holding Register assignment

The default setting of $SNPX_ASG is the following.

Table 3-6. Default setting of $SNPX_ASG variables

System variable	Value
$SNPX_ASG[1].$ADDRESS	1
$SNPX_ASG[1].$SIZE	10000
$SNPX_ASG[1].$VAR_NAME	R[1]@1.1
$SNPX_ASG[1].$MULTIPLY	1

By the default setting, the assignment of Holding Registers is the following.

Table 3-7. Assignment of Holding Registers from default $SNPX_ASG variable settings

Robot data	Address of Holding Registers	Example
Register R[*x*]	*x*	R[1] ⇔ Address 1

Note: Data type is 16 bits signed integer. Value is rounded off to no decimal place. Range of value is from -32768 to 32767. If value is out of the range, value cannot be read and write correctly.

3.3.1 Data type of Holding Registers

Data type of Holding Register is 16 bits integer. Data range of Holding Register is -32768 ~ 32767. The value of assigned robot data is converted as follows according to the data type of the assigned data.

Table 3-8. Holding Register data type conversions

Assigned data	Description
16 bits signed integer Range: -32768 ~ 32767	The Holding Register data is the same as the assigned data.
32 bits signed integer Range: -2147483648 ~ 2147483647	The continued 2 Holding Registers express a 32 bits signed integer. The former address is lower 16 bits data, and the latter address is upper 16bits data.
Real	The continued 2 Holding Registers express a single precision binary real format of IEEE 754. The former address is lower 16 bits data, and the latter address is upper 16bits data. In case of that real type robot data is assigned to Holding Registers, when $MULTIPLY is 0, the robot data is assigned to Holding Registers as real type data. When $MULTIPLY is not 0, the robot data is assigned to Holding Registers as 32 bits signed integer. In this case, the Holding Registers has the value that the robot data is multiplied by $MULTIPLY and rounded off to no decimal place.
String	The continued Holding Registers express a string data. One Holding Register expresses 2 characters. Byte order of string data is specified by $MULTIPLY as follows. $MULTIPLY = 1 (0 or more): Lower byte is the first character, upper byte is the second character. $MULTIPLY = -1 (less than 0): Upper byte is the first character, lower byte is the second character. The last of the string data must be 0. When string data is written to Holding Registers, it is necessary to write 0 to the last of the string data.

⚠ WARNING

If Integer data is written to the Holding Registers of Real data type, the assigned robot data may be set to the value that is too big or cannot be processed. And it may cause serious effect such as the robot moves to unexpected position because the assigned position register has abnormal position data. Please set $MULTIPLY to the value of not 0 normally, and assign 32 bit signed integer data to Holding Registers.

3.3.2 Assign Robot Registers

$SNPX_ASG setting to assign Robot Registers to Holding Registers is the following.

Table 3-9. $SNPX_ASG variable descriptions for Robot Register assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. One Register uses two Holding Registers. (You can change the number of Holding Registers for one robot register by adding "@" in $VAR_NAME) Range: 1~16384
$VAR_NAME	Meaning: String to specify robot data. Please set "R[1]" to assign Register R[1]. The number in [ ] is the index of register. Continued registers such as R[2]-R[5] can be assigned by one $SNPX_ASG element. In this case, please set $SIZE to 8 because the number of registers to assign is 4. And please set $VAR_NAME "R[2]". The index 2 means the starting index of the continuous assignment. If "@1.1" is added just after the string, the number of Holding Registers for one register is changed from 2 to 1. In this case, accessing data size becomes 16 bits. Example: R[1]@1.1 "R[1]" part means to assign robot registers from index 1. "@1.1" part means one register is accessed as 16 bits data.
$MULTIPLY	Meaning: Multiplier The value accessed by an HMI device etc. is multiplied by $MULTIPLY. When $MULTIPLY is 0, it means that an HMI device etc. can access Holding Registers as 32bits REAL data. When it is not 0, an HMI device etc. accesses Holding Registers as 32bits signed integer. And value is rounded off to no decimal place. Range: 0.0001~10000, 0 Example: Register value is 123.45. When $MULTIPLY is 1, Holding Register is 123 When $MULTIPLY is 10, Holding Register is 1235. When $MULTIPLY is 0.1, Holding Register is 12. When $MULTIPLY is 0, Holding Register is 123.45 of REAL

For example, $SNPX_ASG is set as follows.

Table 3-10. Example Robot Register assignments of $SNPX_ASG

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	2	R[1]@1.1	1
$SNPX_ASG[2]	3	4	R[1]	100
$SNPX_ASG[3]	7	4	R[2]	0.1
$SNPX_ASG[4]	11	2	R[1]	0

In this case, address of Holding Registers are corresponded to robot registers as follows.

Table 3-11. Result of example Robot Register assignments of $SNPX_ASG

Address	Assigned data
1	R[1] as 16bits signed integer.
2	R[2] as 16bits signed integer.
3-4	R[1] multiplied by 100 as 32bits signed integer.
5-6	R[2] multiplied by 100 as 32bits signed integer.
7-8	R[2] divided by 10 as 32bits signed integer.
9-10	R[3] divided by 10 as 32bits signed integer.
11-12	R[1] as 32bits REAL

$SNPX_ASG[1] defines that 2 Holding Registers from address 1 to 2 are assigned to robot registers from R[1] multiplied by 1 as 16 bits signed integer. One Holding Register address is 16 bits data, and one register uses one Holding Register. So, address 1 is assigned to R[1] as 16 bits signed integer and address 2 is assigned to R[2] as 16bits signed integer.

$SNPX_ASG[2] defines that 4 Holding Registers from address 3 to 6 are assigned to robot registers from R[1] multiplied by 100 as 32 bits signed integer. One robot register uses 2 Holding Registers. So, addresses 3-4 are assigned to R[1] multiplied by 100 as 32 bits signed integer and address 5-6 are assigned to R[2] multiplied by 100 as 32 bits signed integer.

$SNPX_ASG[3] defines that 4 Holding Registers from 7 to 10 are assigned to robot registers from R[2] divided by 10 as 32 bits signed integer. One robot register uses 2 Holding Registers. So, addresses 7-8 are assigned to R[2] divided by 10 as 32 bits signed integer and address 9-10 are assigned to R[3] divided by 10 as 32 bits signed integer.

$SNPX_ASG[4] defines that 2 Holding Registers from 11 to 12 are assigned to robot register R[1] as 32bit REAL. So, addresses 11 to 12 are assigned to R[1] as 32bits REAL. OPC UA clients can read this bit array as Int16, so the OPC UA client should change this value to REAL number.

3.3.3 Assign Position Registers

$SNPX_ASG setting to assign Position Registers to Holding Registers is the following.

Table 3-12. $SNPX_ASG variable descriptions for Position Register assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. One Position Register uses 50 Holding Registers. (You can change the number of Holding Registers for one position register by adding "@" in $VAR_NAME) Range: 1~16384
$VAR_NAME	Meaning: String to specify robot data. Please set "PR[1]" to assign Position Register PR[1]. The number in [ ] is the index of position register. Continued position registers like PR[2]-PR[5] can be assigned by one $SNPX_ASG element. In this case, please set $SIZE 200 because the number of position registers to assign is 4. And please set $VAR_NAME "PR[2]". The index 2 means the starting index of the continuous assignment. In multi group system, PR[1] means group 1 data of PR[1]. Please set PR[G2:1] to assign group 2 data of PR[1]. If "@" is added just after the string, the specified part of the position register is assigned. This is explained later.
$MULTIPLY	Meaning: Multiplier Only REAL values like X, Y, Z and J1 are effected by this setting. Refer to the table below to see which elements are affected. The value accessed by an HMI device etc. is multiplied by $MULTIPLY. When $MULTIPLY is 0, it means special that an HMI device etc. can access Holding Registers as 32bits REAL data. When it is not 0, an HMI device etc. accesses Holding Registers as 32bits signed

Table 3-12. $SNPX_ASG variable descriptions for Position Register assignment to Holding Registers

	integer. And value is rounded off to no decimal place. Range: 0.0001~10000, 0 Example: Position register member value is 123.45. When $MULTIPLY is 1, Holding Register is 123 When $MULTIPLY is 10, Holding Register is 1235. When $MULTIPLY is 0.1, Holding Register is 12. When $MULTIPLY is 0, Holding Register is 123.45 of REAL

⚠ WARNING

If Integer data is written to the Holding Registers of Real data type, the assigned robot data may be set to the value that is too big or cannot be processed. And it may cause serious effect such as the robot moves to unexpected position because the assigned position register has abnormal position data. Please set $MULTIPLY to the value of not 0 normally, and assign 32 bit signed integer data to Holding Registers.

NOTE

When the position register is locked by LOCK PREG instruction, the writing to the position register by this function is ignored. To confirm if the writing is succeeded or not, please read the position data just after the writing and confirm the read data.

One position register uses 50 Holding Registers. Contents of the 50 Holding Registers are the following.

Table 3-13. Holding Register mapping per Position Register assignment

Address	Description		Effect of $MULTIPLY
Cartesian data			
1-2	X	32bits signed integer or real (mm)	Yes
3-4	Y	32bits signed integer or real (mm)	Yes
5-6	Z	32bits signed integer or real (mm)	Yes
7-8	W	32bits signed integer or real (deg)	Yes

Table 3-13. Holding Register mapping per Position Register assignment

Address	Description		Effect of $MULTIPLY
9-10	P	32bits signed integer or real (deg)	Yes
11-12	R	32bits signed integer or real (deg)	Yes
13-14	E1	32bits signed integer or real (mm, deg)	Yes
15-16	E2	32bits signed integer or real (mm, deg)	Yes
17-18	E3	32bits signed integer or real (mm, deg)	Yes
19	FLIP	16bits signed integer (1:Flip, 0:Non flip)	No
20	LEFT	16bits signed integer (1:Left, 0:Right)	No
21	UP	16bits signed integer (1:Up, 0:Down)	No
22	FRONT	16bits signed integer (1:Front, 0:Back)	No
23	TURN4	16bits signed integer (-128~127)	No
24	TURN5	16bits signed integer (-128~127)	No
25	TURN6	16bits signed integer (-128~127)	No
26	VALIDC	16bits signed integer (→NOTE1)	No
Joint data			
27-28	J1	32bits signed integer or real (mm, deg)	Yes
29-30	J2	32bits signed integer or real (mm, deg)	Yes
31-32	J3	32bits signed integer or real (mm, deg)	Yes
33-34	J4	32bits signed integer or real (mm, deg)	Yes
35-36	J5	32bits signed integer or real (mm, deg)	Yes
37-38	J6	32bits signed integer or real (mm, deg)	Yes
39-40	J7	32bits signed integer or real (mm, deg)	Yes
41-42	J8	32bits signed integer or real (mm, deg)	Yes
43-44	J9	32bits signed integer or real (mm, deg)	Yes
45	VALIDJ	16bits signed integer (→NOTE2)	No
Frame number			
46	UF	16bits signed integer (-1~62) (→NOTE3)	No

Table 3-13. Holding Register mapping per Position Register assignment

Address	Description	Effect of $MULTIPLY
47	UT 16bits signed integer (-1~30) (→NOTE4)	No
48-50	Reserve	No

NOTE

1 VALIDC shows whether the position data is valid for Cartesian or not. This becomes 0 at the following situation, and it becomes 1 in the other situation.
- Position data has uninitialized data (displayed as "******" on Teach Pendant).
- Position data is Joint representation and it can not be converted to Cartesian.

If HMI device etc. writes any data to VALIDC, position representation is changed to Cartesian. In this case, any value is OK to write.

2 VALIDJ shows whether the position data is valid for Joint or not. This becomes 0 at the following situation, and it becomes 1 in the other situation.
- Position data has uninitialized data (displayed "******" on Teach Pendant).
- Position data is Cartesian representation and it can not be converted to Joint.

If HMI device etc. writes any data to VALIDJ, position representation is changed to Joint. In this case, any value is OK to write.

3 UF is user frame number.
If UF is 0, world frame is used.
If UF is -1, the current selected user fame is used.
UF of Position Register is always -1. (In *i*Pendant it is displayed as F)
This value cannot be changed.

4 UT is tool frame number
If UT is 0, mechanical interface frame is used.
If UT is -1, the current selected tool fame is used.
UT of Position Register is always -1. (In *i*Pendant it is displayed as F)
This value cannot be changed.

- Position register has 2 representations, Cartesian or Joint. If detail of the position register is displayed as X,Y,Z,W,P,R, it is Cartesian representation. If it is displayed as J1-6, it is Joint representation.
- An HMI device etc. can always access to any member without regarding to current representation. Position representation is changed by reading from HMI device etc. But if the representation of position register and the representation of accessed member from an HMI device etc. are different, please note the following.
- If the position data is out of stroke limit or there is uninitialized member, this position data cannot be converted to another representation. In this case, the all members in the part of another representation becomes 0.
  For example, a position register is Cartesian representation and X is 10000, it is out of stroke limit, J1-J9 is read as 0 by an HMI device etc. In this case, the members of Cartesian part or UF/UT part can be read correctly.
- If the representation of position register and the representation of accessed member from an HMI device etc. are different, communication response time is increased. Because position representation conversion takes much time. If an HMI device etc. reads many position registers, this conversion time may make big effect to response time.

Uninitialized member of position data is displayed as "******" on position register menu of Teach Pendant. These members are read as 0 from an HMI device etc. Please read VALIDC or VALIDJ to check whether the value is 0 or uninitialized. If position data has uninitialized member, VALIDJ and VALIDC are 0.

If you write data to the member in Cartesian part from an HMI device etc., the position representation becomes Cartesian. If you write data to member in Joint part from an HMI device etc., the position representation becomes Joint. If you write data to UF or UT from an HMI device etc., position representation is not changed.

Extract a part by adding "@" in $VAR_NAME

If "@" is added to $VAR_NAME, the specified part of data structure is extracted. It was already used for Register assignment (R[1]@1.1).

For example, you need to access only X, Y and Z of PR[1-3]. In this case, normally the setting of $SNPX_ASG is the following.

Table 3-14. Example assignment of three Position Register structures

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	150	PR[1]	1

The following is the correspondence between position registers and Holding Registers of this setting.

Table 3-15. Correspondence between Holding Registers and Position Registers from inefficient assignment

Address	Assigned data
1-2	X of PR[1] as 32bits signed integer
3-4	Y of PR[1] as 32bits signed integer
5-6	Z of PR[1] as 32bits signed integer
51-52	X of PR[2] as 32bits signed integer
53-54	Y of PR[2] as 32bits signed integer
55-56	Z of PR[2] as 32bits signed integer
101-102	X of PR[3] as 32bits signed integer
103-104	Y of PR[3] as 32bits signed integer
105-106	Z of PR[3] as 32bits signed integer

Actual read data are only 18 Holding Registers, but this assignment occupies 150 Holding Registers. And, some HMI devices access address 7-50 of Holding Registers that is not necessary to read, because reading one big data is more efficient than reading several small data from the communication point of view. But the address 27-45 includes Joint representation part, and if position data is Cartesian representation, representation conversion is needed to read this unnecessary data.

To communicate efficiently, please set $SNPX_ASG as follows.

Table 3-16. Example of extracting part of three Position Register structures by use of "@" in $VAR_NAME

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	18	PR[1]@1.6	1

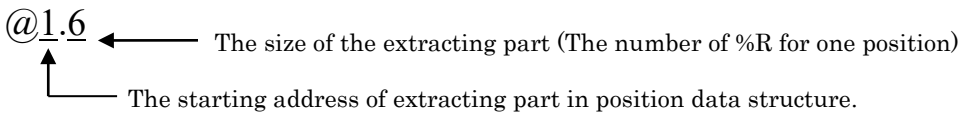
The following is the correspondence between position registers and Holding Registers of this setting.

Table 3-17. Correspondence between Holding Registers and Position Registers from efficient assignment

Address	Accessed data
1-2	X of PR[1] as 32bits signed integer
3-4	Y of PR[1] as 32bits signed integer
5-6	Z of PR[1] as 32bits signed integer
7-8	X of PR[2] as 32bits signed integer
9-10	Y of PR[2] as 32bits signed integer
11-12	Z of PR[2] as 32bits signed integer
13-14	X of PR[3] as 32bits signed integer
15-16	Y of PR[3] as 32bits signed integer
17-18	Z of PR[3] as 32bits signed integer

The change of $SNPX_ASG is that "@1.6" is added in $VAR_NAME and $SIZE is changed from 150 to 18. By this change, the number of Holding Registers for one position register is changed from 50 to 6. The "6" in "@1.6" means the number of Holding Registers for one position register. And the "1" in "@1.6" means the starting address of extracting part in position data structure.

By specifying "@" in $VAR_NAME, you can extract the specified part of position data structure.



You can specify "@" not only for position register, but also for all data that is assigned by $SNPX_ASG.

The following is another example to assign J1-J6 of PR[1-3].

Table 3-18. Example Position Register assignment of J1-J6 of PR[1-3]

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	96	PR[3]@27.12	1

3.3.4 Assign String Registers

$SNPX_ASG setting to assign String Registers to Holding Registers is the following.

Table 3-19. $SNPX_ASG variable descriptions for String Register assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. One String Register uses 40 Holding Registers. It is possible to deal with first 80 characters of one string register in default setting. (You can change the number of Holding Registers for one string register by adding "@" in $VAR_NAME) Range: 1~16384
$VAR_NAME	Meaning: String to specify robot data. Please set "SR[1]" to assign String Register SR[1]. The number in [ ] is the index of string register. Continued string registers such as SR[2]-SR[5] can be assigned by one $SNPX_ASG element. In this case, please set $SIZE 160 because the number of string registers to assign is 4. And please set $VAR_NAME "SR[2]". The index 2 means the starting index of the continuous assignment. If "@" is added just after the string, the specified part of the String Register is assigned. Example: SR[1]@1.50 "SR[1]" means to assign String Registers from index 1. "@1.50" means the first 100 characters of String Register data are assigned.
$MULTIPLY	Meaning: Specify byte order of string. Range: 1,-1

3.3.5 Assign Current Position

$SNPX_ASG setting to assign Current Position to Holding Registers is the following.

Table 3-20. $SNPX_ASG variable descriptions for Current Position assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. Current Position uses 50 Holding Registers. (You can change the number of Holding Registers for Current Position by adding "@" in $VAR_NAME) Range: 1~16384

Table 3-20. $SNPX_ASG variable descriptions for Current Position assignment to Holding Registers

Variable in $SNPX_ASG	Description
$VAR_NAME	Meaning: String to specify robot data. Please set ”POS[0]” to assign Current Position. The number in [ ] is user frame number. When user frame number is 0, the Current Position on World frame is assigned. This is the same as when "WORLD" is selected in the current position screen on the teaching pendant. When user frame number is -1, the Current Position on the User frame that is currently selected. This is the same as when "USER" is selected in the current position screen on the teaching pendant. When user frame number is 1-61, the Current Position on the specified user frame. Data structure is the same as position register. In multi group system, POS[0] means current position group 1 robot. Please set POS[G2:0] to assign current position of group 2 robot. If “@” is added just after the string, the specified part of current position is assigned.
$MULTIPLY	Meaning: Multiplier Only REAL values like X, Y, Z and J1 are effected by this setting. Refer to the table below to see which elements are affected. The value of each element of the current position multiplied by the value of $MULTIPLY is read and written. If $MULTIPLY is 0, it means that the data is assigned to Holding Registers as 32bits REAL data. If it is not 0, it means that the data is assigned to Holding Registers as 32bits signed integer. And value is rounded off to no decimal place. Range: 0.0001~10000, 0 Example: Current position member value is 123.45. When $MULTIPLY is 1, Holding Register is 123 When $MULTIPLY is 10, Holding Register is 1235. When $MULTIPLY is 0.1, Holding Register is 12. When $MULTIPLY is 0, Holding Register is 123.45 of REAL

NOTE

Current position is read only. If you write it, nothing occurs.

Current position uses 50 Holding Registers, and data structure is the same as position register.

UT of current position is always -1.

UF of current position is the specified user frame number in $VAR_NAME.

Current position is not assigned continuously. For example, the following setting of $SNPX_ASG assigns current position of user frame 1 to address 1-50, but current position of user frame 2 is not assigned to address 51-100. Please set "POS[2]" in another $SNPX_ASG.

Table 3-21. Example of discontinuous assignment of Current Position

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	100	POS[1]	1

3.3.6 Assign Alarm History

$SNPX_ASG setting to assign Alarm history to Holding Registers is the following.

Table 3-22. $SNPX_ASG variable descriptions for Alarm History assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. One alarm uses 100 Holding Registers. (You can change the number of Holding Registers for one alarm by adding "@" in $VAR_NAME) Range: 1~16384

Table 3-22. $SNPX_ASG variable descriptions for Alarm History assignment to Holding Registers

Variable in $SNPX_ASG	Description
$VAR_NAME	Meaning: String to specify robot data. Please set ”ALM[1]” to assign alarm. The number in [ ] is line number in alarm menu. The latest alarm is 1. “ALM[1]” assigns active alarm. Alarms displayed in active alarm menu are assigned. “ALM[E1]” assigns alarm history. Alarms displayed in alarm history menu are assigned. “ALM[M1]” assigns motion alarm history. Alarm displayed in motion alarm menu are assigned. “ALM[S1]” assigns system alarm history. Alarm displayed in system alarm menu are assigned. “ALM[A1]” assigns application alarm history. Alarm displayed in application alarm menu are assigned. “ALM[P1]” assigns password log. Password log displayed in password log menu are assigned.
$MULTIPLY	Meaning: Specify byte order of string. Range: 1,-1

NOTE

Alarm history is read only. If you write it, nothing occurs.

One alarm uses 100 Holding Registers. Contents of the 100 Holding Registers are the following.

Table 3-23. Holding Register content per Alarm assignment

Address	**Description**
1	Facility code 16 bits signed integer When alarm is “SRVO-001”, facility code is 11 that means “SRVO”. Please refer the "2.2.2 Facility Name and Code" in "OPERATOR'S MANUAL (Alarm Code List)" (B-83284EN-1) about the number of facility code.
2	Alarm number 16 bits signed integer When alarm is “SRVO-001”, alarm number is 1.
3	Facility code of Cause Code 16bits signed integer

Table 3-23. Holding Register content per Alarm assignment

	When alarm occurs, alarm message is displayed on top of teach pendant. Sometimes a message is also displayed on the second line, it is cause code. This shows facility code of the cause code. If the alarm does not have cause code, this becomes 0.
4	Alarm number of cause code 16 bits signed integer Alarm number of cause code. If the alarm does not have cause code, this becomes 0.
5	Alarm severity 16 bits signed integer The value shows alarm severity. NONE 128 WARN 0 PAUSE.L 2 PAUSE.G 34 STOP.L 6 STOP.G 38 SERVO 54 ABORT.L 11 ABORT.G 43 SERVO2 58 SYSTEM 123
6	Occurred Time (year) 16 bits signed integer
7	Occurred Time (month) 16 bits signed integer
8	Occurred Time (day) 16 bits signed integer
9	Occurred Time (hour 24 hour system) 16 bits signed integer
10	Occurred Time (minutes) 16 bits signed integer
11	Occurred Time (second) 16 bits signed integer
12-51	Alarm message string of up to 80 characters Alarm message string can be read. It shows the string as the same as that is displayed on top line of teach pendant such as “SRVO-001”.
52-91	Cause code alarm message string of up to 80 characters Alarm message of cause code.
92-100	Alarm severity word 80 characters string String of alarm severity such as “WARN”.

Facility code and alarm number of “RESET” is read as 0. Alarm message is “RESET”.

When there is only 2 lines on active alarm menu, all members of alarms after ALM[3] are 0.

In string item, addresses after string are 0.

One alarm uses 100 Holding Registers, if you would like to read only facility code and alarm number, most of 100 Holding Registers area is not used. To communicate efficiently, please use "@".

Example: $SNPX_ASG is set as follows.

Table 3-24. Example for efficient reading of only facility code and alarm number

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	12	ALM[E1]@1.4	1

The correspondence of address and alarm data of this setting is the following.

Table 3-25. Result of efficient alarm assignment

Address	Assigned data
1	Alarm ID of alarm1
2	Alarm number of alarm1
3	Alarm ID of cause code of alarm1
4	Alarm number of cause code of alarm1
5	Alarm ID of alarm2
6	Alarm number of alarm2
7	Alarm ID of cause code of alarm2
8	Alarm number of cause code of alarm2
9	Alarm ID of alarm3
10	Alarm number of alarm3
11	Alarm ID of cause code of alarm3
12	Alarm number of cause code of alarm3

3.3.7 Assign Program Execution Status

$SNPX_ASG setting to assign program execution status to Holding Registers is the following.

Table 3-26. $SNPX_ASG variable descriptions for Program Execution Status assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. One task uses 18 Holding Registers. (You can change the number of Holding Registers for one task by adding "@" in $VAR_NAME) Range: 1~16384
$VAR_NAME	Meaning: String to specify robot data. Please set "PRG[1]" to assign program execution status. The number in [ ] is task number. In single task system, program execution status always can be assigned by PRG[1]. In multi task system, when 2 tasks are running at the same time, PRG[1] and PRG[2] shows the execution status of each task. Which task is PRG[1] is decided by timing of execution and communication. But the task that is read as PRG[1] is always read as PRG[1] until it is aborted.
$MULTIPLY	Meaning: Specify byte order of string. Range: 1,-1

NOTE

Program execution status is read only. If you write it, nothing occurs.

One task uses 18 Holding Registers. Contents of the 18 Holding Registers are the following.

Table 3-27. Holding Register content per task execution assignment

Address	Description
1-8	Program name 16 characters string Name of running program. When sub program is called, it shows sub program name.
9	Line number 16bits signed integer. Execution line number. When sub program is called, it shows line number of sub program.

Table 3-27. Holding Register content per task execution assignment

Address	Description
10	Execution status 16bits signed integer Aborted 0 Paused 1 Running 2
11-18	Parent program name 16 characters string Name of the started program When sub program is not called, it is the same as program name.

When program is aborted, all members become 0.

By assigning each strings to $VAR_NAME, the type of the program can be selected. And the meaning of line number is changed depending on the situation.

Table 3-28. Syntax examples for access of specific type of program and line number access

$VAR_NAME	Program name and line number
PRG[1]	Program name Name of running program. Line number Execution line number.
PRG[M1]	Program name When running program is MACRO program, it shows name of parent program that calls the running program. When the parent program is also MACRO program, it shows name of parent program that is not MACRO program. Line number Line number of CALL instruction that calls the running program. When all parent programs are MACRO program. It becomes 0.
PRG[K1]	Program name When running program is KAREL program, it shows name of parent program that calls the running program. When the parent program is also KAREL program, it shows name of parent program that is not KAREL program. Line number Line number of CALL instruction that calls the running program. When all parent programs are KAREL program. It becomes 0.

Table 3-28. Syntax examples for access of specific type of program and line number access

PRG[MK1] or PRG[KM1]	Program name When running program is MACRO or KAREL program, it shows name of parent program that calls the running program. When the parent program is also MACRO or KAREL program, it shows name of parent program that is not MACRO nor KAREL program. Line number Line number of CALL instruction that calls the running program. When all parent programs are MACRO or KAREL program. It becomes 0.

By adding "@", a part of this structure is assigned to Holding Registers.

Example: $SNPX_ASG is set as follows.

Table 3-29. Example of efficient access of specific Program Execution Status structure

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	4	PRG[1]@9.2	1

The correspondence of address and program execution status of this setting is the following.

Table 3-30. Result of efficient Program Execution Status structure access

Address	Accessed data
1	Line number of task 1
2	Execution status of task 1
3	Line number of task 2.
4	Execution status of task 2.

3.3.8 Assign System Variables

$SNPX_ASG setting to assign system variables to Holding Registers is the following.

Table 3-31. $SNPX_ASG variable descriptions for System Variable assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. The number of Holding Registers for one system variable is defined by data type of the system variable. Please refer the following data type list. (You can change the number of Holding Registers for one system variable by adding "@" in $VAR_NAME) Range: 1~16384
$VAR_NAME	Meaning: String to specify robot data. Please set system variable name, for example "$SNPX_ASG[1].$ADDRESS". If array system variable such as "$SNPX_ASG[1]" is set, array elements are assigned continuously like Registers. KAREL string can be specified by the following format. $[KAREL program name]variable name.
$MULTIPLY	Meaning: The meaning of $MULTIPLY is defined according to the data type of system variable. Please refer the following data type list.

To access to system variable, data type of assigned Holding Registers and meaning of $MULTIPLY are changed according to data type of system variable. The following is the list of data type of system variables that can be assigned to Holding Registers, and it also explain the number of Holding Registers and meaning of $MULTIPLY.

Table 3-32. Listing of supported data types of system variables for access through Holding Registers

Data type of system variables	The number of Holding Registers for one system variable	Meaning of $MULTIPLY
INTEGER 32bits signed integer	2	The data is multiplied by $MULTIPLY. The data is accessed as 32 bits signed integer. And value is

Table 3-32. Listing of supported data types of system variables for access through Holding Registers

SHORT 16bits signed integer	2	rounded off to no decimal place. If $MULTIPLY is 0, it is same as $MULTIPLY is 1.
BYTE 8bits signed integer	2	
REAL 32bits real	2	The data is multiplied by $MULTIPLY. If $MULTIPLY is 0, it means that the data is accessed as 32 bits REAL data. If it is not 0, the data is accessed as 32 bits signed integer. And value is rounded off to no decimal place
BOOLEAN TRUE/FALSE	2	$MULTIPLY is not used for this data type. The data is accessed as 32bit signed integer. If value is TRUE, Holding Register is 1. If it is 0, Holding Register is 0. If 0 is written, the system variable becomes FALSE. If not 0 is written, the system variable becomes TRUE.
POSITION Position data	50	Data structure is the same as position register. Meaning of $MULTIPLY is also same as position register.
STRING String data	40	Specify byte order of string.

INTEGER, SHORT and BYTE are accessed by the same way. The system variable that integer value is displayed in system variable menu on teach pendant is INTEGER, SHORT or BYTE.

The variable that real value is displayed in system variable menu on teach pendant is REAL.

The variable that TRUE or FALSE is displayed in system variable menu on teach pendant is BOOLEAN.

The variable that string is displayed on system variable menu on teach pendant is STRING.

The variable that is displayed as POSITION in system variable menu on teach pendant is POSITION.

⚠ WARNING

The system variable that is protected in system variable menu on teach pendant can be changed from Holding Registers. If illegal value is set to system variable, system may be harmed seriously. Please check value enough to write system variables.

⚠ WARNING

If Integer data is written to the Holding Registers of Real data type, the assigned robot data may be set to the value that is too big or cannot be processed. And it may cause serious effect such as the robot moves to unexpected position because the assigned POSITION type variable has abnormal position data. Please set $MULTIPLY to the value of not 0 normally, and assign 32 bit signed integer data to Holding Registers.

By adding "@", a part of this structure is assigned to Holding Registers.

For example. the following $SNPX_ASG assigns X, Y and Z of user frame 1 and 2 of group 1.

Table 3-33. Example of efficient access of System Variable structure

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	12	$MNUFRAME[1,1]@1.6	1

This assigns system variables to Holding Registers as follows.

Table 3-34. Result of Efficient System Variable structure access

Address	Accessed data
1-2	X of user frame 1 of Group 1 (X of $MNUFRAME[1,1])
3-4	Y of user frame 1 of Group 1 (Y of $MNUFRAME[1,1])
5-6	Z of user frame 1 of Group 1 (Z of $MNUFRAME[1,1])
7-8	X of user frame 2 of Group 1 (X of $MNUFRAME[1,2])
9-10	Y of user frame 2 of Group 1 (Y of $MNUFRAME[1,2])
11-12	Z of user frame 2 of Group 1 (Z of $MNUFRAME[1,2])

3.3.9 Assign comment

$SNPX_ASG setting to assign comment of Register, Position register, String register and I/O to Holding Registers is the following.

Table 3-35. $SNPX_ASG variable descriptions for Comment assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1 to 16384
$SIZE	Meaning: Number of Holding Registers to assign. One comment uses 40 Holding Registers. Please set according to the number of comments to assign. (You can change the number of Holding Registers for one comment by adding "@" in $VAR_NAME) Range: 1 to 16384

Table 3-35. $SNPX_ASG variable descriptions for Comment assignment to Holding Registers

Variable in $SNPX_ASG	Description
$VAR_NAME	Meaning: String to specify robot data. Please set ”R[C1]” to assign comment of register. The number in [ ] is index of register. To assign comment of position register, string register and I/O, please set the following string. Register R[C1] Position register PR[C1] String register SR[C1] DI DI[C1] DO DO[C1] RI RI[C1] RO RO[C1] UI UI[C1] UO UO[C1] SI SI[C1] SO SO[C1] WI WI[C1] WO WO[C1] WSI WSI[C1] WSO WSO[C1] GI GI[C1] GO GO[C1] AI AI[C1] AO AO[C1] Comment of the continued registers such as R[2]~R[5] can be assigned by one $SNPX_ASG element. In this case, please set $SIZE 160 because the number for one comment to assign is 40. And please set $VAR_NAME “R[C2]”. The index 2 means the starting index of the continuous assignment. If “@” is added just after the string, the specified part of the comment is assigned. Example: R[C1]@1.5 “R[C1]” means to assign comment of registers from index 1. “@1.5” means the first 10 characters of comment are assigned.
$MULTIPLY	Meaning: Specify byte order of string. Range: 1,-1

3.3.10 Assign I/O data and simulation status

$SNPX_ASG setting to assign I/O data and simulation status to Holding Registers is the following.

Input signal can’t be changed from this function even though it is assigned to Holding Registers.

If you want to change input signal from this function, there is a way to change input signal through flag with assigning the input signal to flag, which is rack 34 and slot 1.

Table 3-36. $SNPX_ASG variable descriptions for I/O data and Simulation Status assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. The number of Holding Registers for one I/O value or simulation status is changed by $MULTIPLY setting. When $MULTIPLY is 1, one I/O value or simulation status uses one Holding Register. When $MULTIPLY is 0, one Holding Register includes 16 I/O value or simulation status as bit image. (Note: Value of GI/O and AI/O use one Holding Register even if $MULTIPLY is 0.) Range: 1~16384

Table 3-36. $SNPX_ASG variable descriptions for I/O data and Simulation Status assignment to Holding Registers

Variable in $SNPX_ASG	Description
$VAR_NAME	Meaning: String to specify robot data. Please set "DI[1]" to assign DI[1] data. The number in [ ] is index of DI. To assign I/O data or simulation status of the other type of I/O, please set the following string. DI: Value DI[1], Simulation DI[S1] DO: Value DO[1], Simulation DO[S1] RI: Value RI[1], Simulation RI[S1] RO: Value RO[1], Simulation RO[S1] UI: Value UI[1] UO: Value UO[1] SI: Value SI[1] SO: Value SO[1] WI: Value WI[1], Simulation WI[S1] WO: Value WO[1], Simulation WO[S1] WSI: Value WSI[1], Simulation WSI[S1] WSO: Value WSO[1], Simulation WSO[S1] GI: Value GI[1], Simulation GI[S1] GO: Value GO[1], Simulation GO[S1] AI: Value AI[1], Simulation AI[S1] AO: Value AO[1], Simulation AO[S1] Flag: F[1] Marker: M[1] Example: Assign DI[11]~DI[42]: When $MULTIPLY is 1, please set 32 to $SIZE because the number of DI is 32, and set "DI[11]" to $VAR_NAME. In this case, index 11 means that the start index of the assignment is 11. When $MULTIPLY is 0, 16 DI data are assigned to one Holding Register, so please set 2 to $SIZE, and set "DI[11]" to $VAR_NAME.
$MULTIPLY	Meaning: One Holding Register has one I/O data or 16 I/O data as bit image. One I/O data is assigned to one Holding Register 1 16 I/O data are assigned to one Holding Register 0

You can select by $MULTIPLY setting whether one I/O data is assigned to one Holding Register or 16 I/O data are assigned to one Holding Register.

When $MULTIPLY is 1, one I/O data or one simulation status are assigned to one Holding Register. When the value is ON, the corresponded Holding Register is set to 1. When the value is OFF, the corresponded Holding Register is set to 0.

When $MULTIPLY is 0, 16 I/O data or 16 simulation status are assigned to one Holding Register. When the value is ON, the corresponded bit of Holding Register is set to 1. When the value is OFF, the corresponded bit of Holding Register is set to 0. I/O of lower index is assigned to lower bit of Holding Register.

Example: When DI[1-16] is assigned to Holding Register, if only DI[1] is ON and the others are OFF, the Holding Register is 1. If only DI[16] is ON and the others are OFF, the Holding Register is 32768 (in case of unsigned 16 bit integer).

3.3.11 Assign Integrated PMC address data

$SNPX_ASG setting to Integrated PMC address data to Holding Registers is the following.

Table 3-37. $SNPX_ASG variable descriptions for PMC Address Data assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. The number of Holding Registers corresponded to PMC address is different by the specified PMC address that is byte address or bit address. When bit address is specified, one bit PMC address is assigned to one Holding Register. When byte address is specified, two bytes PMC addresses are assigned to one Holding Register. Range: 1~16384
$VAR_NAME	Meaning: String to specify robot data. Set ”PMC1:X0” to assign data of PMC address X0. The "PMC1" part is PMC path number. "PMCS" means DCS Safety PMC. Data of bit address is assigned by specifying bit address such as ”PMC1:X0.0”. Example: Assign X0.0~X1.7 data of 1st path PMC When $SIZE is set to 1 and $VAR_NAME is set to ”PMC1:X0”, two byte address X0~X1 are assigned to one Holding Register address. Lower byte is assigned to X0, upper byte is assigned to X1. When $SIZE is set to 16 and $VAR_NAME is set to ”PMC1:X0.0”, 16 bit address X0.0~X1.7 are assigned to 16 Holding Registers. X0.0 is assigned to Address 1, X0.1 is assigned to Address 2, X1.7 is assigned to Address 16.
$MULTIPLY	Not used

NOTE
1 When K900.4 (Change address value) is 0, writing to PMC address data is ignored. To check if the writing is ignored or not, please read the PMC address after writing and confirm the read value.
2 Address of DCS safety PMC is read only. If you write it, nothing occurs.

Example: $SNPX_ASG is set as follows.

Table 3-38. Example of PMC Address Data assignment

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	2	PMC1:R0.0	1
$SNPX_ASG[2]	3	2	PMC1:D0	1

The correspondence between Holding Registers and robot data by this setting is the following.

Bit address is assigned to the address 1 - 2, therefore 1 bit of PMC address is assigned to 1 Holding Register continuously.

Byte address is assigned to the address 3 - 4, therefore 2 bytes of PMC address is assigned to 1 Holding Register continuously.

Table 3-39. Result of PMC Address Data assignment

Address	Assigned robot data
1	R0.0
2	R0.1
3	D0 - D1 (Lower byte:D0, Upper byte:D1)
4	D2 - D3 (Lower byte:D2, Upper byte:D3)

3.3.12 Assign Symbol and Comment of Integrated PMC address

$SNPX_ASG setting to symbol and comment of Integrated PMC address to Holding Registers is the following.

Table 3-40. $SNPX_ASG variable descriptions for Symbol and Comment of Integrated PMC Address assignment to Holding Registers

Variable in $SNPX_ASG	Description
$ADDRESS	Meaning: Start address of Holding Registers to assign. Range: 1~16384
$SIZE	Meaning: Number of Holding Registers to assign. One symbol or comment uses 40 Holding Registers. Please set according to the number of comments to assign. (You can change the number of Holding Registers for one symbol and comment by adding “@” in $VAR_NAME.) Range: 1~16384
$VAR_NAME	Meaning: String to specify robot data. Set ”PMC1:S:X0” to assign symbol of PMC address X0. Set ”PMC1:C:X0” to assign comment of the PMC address. The "PMC1" part is PMC path number. "PMCS" means DCS Safety PMC. Symbol and comment of bit address is assigned by specifying bit address such as ”PMC1:S:X0.0”. Example: PMC1:S:X0.0@1.5 The “PMC1:S:X0.0” part means that symbol of bit address from X0.0 is assigned continuously. The @1.5 part means that the top 10 characters are assigned.
$MULTIPLY	Meaning: Specify byte order of string. Range: 1,-1

NOTE
Symbol and comment of Integrated PMC are read only. If you write it, nothing occurs.

Example: $SNPX_ASG is set as follows.

Table 3-41. Example of Symbol and Comment of Integrated PMC Address access

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	10	PMC1:S:R0.0@1.5	1
$SNPX_ASG[2]	11	10	PMC1:C:D0@1.5	1

The correspondence between Holding Registers and robot data by this setting is the following.

Table 3-42. Resulting Symbol and Comment of Integrated PMC Address access

Address	Assigned robot data
1-5	Symbol of R0.0
6-10	Symbol of R0.1
11-15	Comment of D0
16-20	Comment of D1

3.3.13 Hints

Assigned data is not read correctly

If an HMI device etc. reads a Holding Register that is not assigned to any data, this Holding Register is always read as 0. If Holding Register that the problem occurs is not 0, this Holding Register is assigned to the other data.

For example, when $SNPX_ASG is set as follows, addresses 101-150 of Holding Registers are assigned by both $SNPX_ASG[1] and $SNPX_ASG[2].

Table 3-43. Hints: Example of overlapping holding register assignment

	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
$SNPX_ASG[1]	1	1000	R[1]@1.1	1
$SNPX_ASG[2]	101	50	PR[1]	100

In this case, the $SNPX_ASG whose index is smaller is used.

Therefore, addresses 101-150 are assigned to R[101]-R[150], and PR[1] can not be accessed by an HMI device etc.

If the read data of Holding Register is 0, there may also be duplicated assignment as above. Please check duplication of $SNPX_ASG.

If $SNPX_ASG setting has problem even though assignment is not duplicated, please check $VAR_NAME of $SNPX_ASG.

The following list shows the correct format of $VAR_NAME setting. If the setting is not match to them, robot data is not assigned.

Table 3-44. Hints: Listing of valid $VAR_NAME setting

Correct format	Note
“R[n]”	
“PR[n]” “PR[Gn:n]	If you specify group number, “:” must be specified between group number and index.
“SR[n]”	
“POS[n]” “POS[Gn:n]	If you specify group number, “:” must be specified between group number and index.
“ALM[n]” “ALM[En]” “ALM[Pn]	Alarm line number must be specified just after E, M, A, S or P. The ":" must not be specified.
“PRG[n]” “PRG[Mn]”, “PRG[Kn]” “PRG[MKn]”, “PRG[KMn]”	Index must be specified just after M, K, MK or KM. The ":" must not be specified.
System variable name	The first character of system variable must be “$”.
$[KAREL program name]variable name	Specify KAREL program name just after ‘$[‘, then specify ‘]’ and specify KAREL variable name just after ‘]’.
“DI[n]”, “DI[Sn]”, “DI[Cn]” “DO[n]”, “DO[Sn]”, “DO[Cn]” “RI[n]”, “RI[Sn]”, “RI[Cn]”, “RO[n]”, “RO[Sn]”, “RO[Cn]” “UI[n]”, “UI[Cn]” “UO[n]”, “UO[Cn]” “SI[n]”, “SI[Cn]” “SO[n]”, “SO[Cn]” “WI[n]”, “WI[Sn]”, “WI[Cn]”, “WO[n]”, “WO[Sn]”, “WO[Cn]”	Index must be specified just after S or C. The ":" must not be specified.

Table 3-44. Hints: Listing of valid $VAR_NAME setting

“WSI[n]”, “WSI[Sn]”, “WSI[Cn]” “WSO[n]”, “WSO[Sn]”, “WSO[Cn]” “GI[n]”, “GI[Sn]”, “GI[Cn]” “GO[n]”, “GO[Sn]”, “GO[Cn]” “AI[n]”, “AI[Sn]”, “AI[Cn]” “AO[n]”, “AO[Sn]”, “AO[Cn]” “F[n]”, “F[Cn]” “M[n]”, “M[Cn]”	
PMC<path>:address PMC<path>:address.bit PMC<path>:S:address PMC<path>:S:address.bit PMC<path>:C:address PMC<path>:C:address.bit	The <path> is from 1 to 5 or "S". The ":" must be specified just after PMC path. And PMC address must be specified just after the ":". For symbol or comment, ":S" or ":C" must be specified just after PMC path.

- If there is space character in $VAR_NAME, data is not assigned. If you specify “@”, please do not add any space before “@”.
- If you use continuous array assignment, the number of Holding Registers for one element is defined according to data type, please check explanation of the assigned robot data.
- The format of “@” is “@n.n”. A “.” must be specified between starting address and the number of Holding Registers. A “@” must be specified just after variable name.

Method to improve response time

- Position representation conversion takes much time. If you access position register, please assign only necessary members of position structure by using “@”. And please change position representation of the position registers to the representation that is accessed by an HMI device etc.

4 DYNAMIC ASSIGNMENT TO HOLDING REGISTERS

Setting $SNPX_ASG is necessary to read or write except for I/O signal. This setting is done by the system variable screen normally. However, a setting which is equivalent to $SNPX_ASG is available for each connection.

Please note that this setting is cleared when the connection is closed.
Settings are enabled only in the same connection so read and write Holding Registers with keeping connection.
This function is supported from 7DF1 series P19 or after and all versions after 7DF1 series.

When 128 characters or less are written to the address 16385 or after of Holding Registers (It is necessary to start writing from address 16385) or Command node, robot regards the written characters as command and processes it. Byte order of string data can be changed with $SNPX_PARAM.$CMD_ENDIAN.
Other than 0 (default) : Lower byte is the first character, upper byte is the second character.
0 : Upper byte is the first character, lower byte is the second character.

The following commands can be executed.
Each command should be written in 1 time. Write some times with segment is invalid.

CLRASG Clear all assignment and initialization
Format: CLRASG
Function: Clear all assignment and initialization.

Do once before following SETASG command sequence.

SETASG Set assignment
Format: SETASG ($ADDRESS) ($SIZE) ($VAR_NAME) [($MULTIPLY)]
Function: Set assignment.

The specification of $ADDRESS, $SIZE, $VAR_NAME and $MULTIPLY are same with the specification of the same variables under $SNPX_ASG.
$MULTIPLY can be abbreviated. If abbreviated, $MULTIPLY is set to 1.
Do CLRASG before executing SETASG sequence.

Example: If assignments are set as following

Table 3-45. Example of desired dynamic assignment to Holding Registers

No.	$ADDRESS	$SIZE	$VAR_NAME	$MULTIPLY
1	1	2	R[1]@1.1	1
2	3	4	R[1]	100
3	7	4	R[2]	0.1
4	11	2	R[1]	0

Write following strings in order:

```
CLRASG
SETASG 1 2 R[1]@1.1 1
SETASG 3 4 R[1] 100
SETASG 7 4 R[2] 0.1
SETASG 11 2 R[1] 0
```

Figure 4-1. Example string order for dynamic assignment of Holding Registers

If some commands are separated with CR (13) or LF (10), some command can be written in 1 sequence but the number of max character in 1 sequence is 128.